Environmental Statistics

Lecture 13: Plotting and Variable importance

Ikedichi Edward Azuh Carsten F. Dormann

1. Overview

Modern machine-learning models, particularly tree-based ensemble models such as Random Forests and Gradient Boosting, can capture complex nonlinear relationships and interactions between predictors. However, this flexibility makes their predictions difficult to interpret because the relationship between individual predictors and model predictions is often not directly visible.

In traditional statistical models, predictor effects are often summarized using estimated parameters. For example, in a linear regression model, regression coefficients describe the expected change in the response associated with a change in a predictor, under the assumptions of the fitted model.

However, complex models such as Random Forests, Gradient Boosting, and Neural Networks do not provide simple coefficients. Their predictions are generated through many nonlinear transformations and interactions between variables.

Therefore, an important question after fitting a predictive model is:

How can we understand what the model has learned?

This lecture introduces methods from explainable machine learning (xAI) that help visualize and quantify predictor effects.

The main objectives are:

- understand why complex models require specialized interpretation tools;
- visualize nonlinear relationships between predictors and predictions;
- introduce Partial Dependence Plots (PDP), also known as marginal effect plots, Individual Conditional Expectation (ICE) plot and Accumulated Local Effects (ALE) plot;
- distinguish between conditional effects and marginal effects;
- quantify predictor importance using model-specific and model-agnostic approaches;
- understand the role of uncertainty when interpreting machine-learning models.

The goal is not only to build accurate models, but also to understand why these models produce their predictions.

2. Why interpretation becomes difficult in machine-learning models

Traditional statistical models are often interpreted through model parameters. For example, in a linear regression model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \epsilon,$$

the coefficient β_j describes the expected change in the response associated with a one-unit increase in predictor X_j, assuming the other predictors remain constant.

However, modern machine-learning models such as Random Forests and boosting algorithms do not describe relationships using a small number of coefficients. Instead, predictions are generated through many nonlinear decision rules and interactions between predictors.

For example, an environmental response may depend on interactions between predictors such as:

$$\{\text{rainfall}, \text{temperature}, \text{soil properties}, \text{topography}\},$$

where the effect of one predictor depends on the values of the others.

A complex model can therefore be represented as:

$$Y = f(X_1, X_2, \dots, X_p),$$

where $f(\cdot)$ is an unknown nonlinear function.

Although such models can provide accurate predictions, the relationship between individual predictors and the response is no longer directly visible.

For example, a tree ensemble may learn that:

$$\text{Effect of rainfall} \neq \text{constant},$$

but instead changes depending on catchment characteristics or climate conditions.

Therefore, interpretation requires methods that simplify a high-dimensional model into visual or quantitative summaries.

These methods do not replace the fitted model. Instead, they provide ways to understand how the model generates its predictions.

3. Visualizing predictor effects

Machine-learning models often involve many predictors and complex interactions. The complete relationship between predictors and model predictions is therefore high-dimensional and cannot usually be visualized directly.

For example, with three predictors,

$$Y = f(X_1, X_2, X_3),$$

the fitted model represents a three-dimensional prediction surface. As the number of predictors increases, the prediction surface becomes impossible to display directly.

A practical solution is to examine the model by taking slices through this high-dimensional space.

This involves selecting one predictor of interest and studying how model predictions change while the remaining predictors are either fixed or summarized in some way.

For a selected predictor X_j, the prediction can be expressed as

$$\hat{Y} = f(X_j, \mathbf{X}_{-j}),$$

where $\mathbf{X}_{-j}$ denotes all remaining predictors.

Several complementary methods have been developed to visualize predictor effects. They differ mainly in how they treat the remaining predictors $\mathbf{X}_{-j}$:

- **Conditional effect plots**: fix the remaining predictors at representative values (for example, their medians) and examine how predictions change as X_j varies.

- **Individual Conditional Expectation (ICE) plots**: fix the remaining predictors at the observed values of each individual observation, producing one prediction curve per observation.
- **Partial Dependence Plots (PDPs)**: average the collection of ICE curves to summarize the overall relationship between X_j and the model prediction.
- **Accumulated Local Effects (ALE) plots**: estimate the average local effect of X_j by accumulating prediction changes within regions supported by the observed predictor distribution.

Together, these methods provide complementary views of how individual predictors influence model predictions.

They can help answer several important questions:

- Is the relationship positive or negative?
- Is the relationship linear or nonlinear?
- Are there threshold effects?
- How much does the predictor influence model predictions?
- How much uncertainty is associated with the estimated effect?

However, these plots describe the behaviour of the fitted model rather than causal relationships. They explain how the model uses the predictors to make predictions, but they do not necessarily reveal the true underlying environmental mechanisms.

3.1 Conditional effect plots

A conditional effect plot (a.k.a "partial effect plot") examines the relationship between one predictor and the model prediction while keeping all other predictors fixed.

Suppose we are interested in predictor X_j. The remaining predictors are represented by:

$$\mathbf{X}_{-j} = (X_1, \dots, X_{j-1}, X_{j+1}, \dots, X_p)$$

A conditional effect plot evaluates:

$$\hat{Y} = f(X_j, \mathbf{X}_{-j}),$$

where $\mathbf{X}_{-j}$ is fixed at selected values.

A common approach is to set all remaining predictors to their typical values, such as their median values:

$$\mathbf{X}_{-j} = \text{median}(\mathbf{X}_{-j})$$

The resulting plot shows how the model prediction changes when only X_j varies.

For example, in a flood prediction model, we may examine the effect of rainfall by fixing all other catchment characteristics:

$$\text{Flood prediction} = f(\text{rainfall}, \text{median area}, \text{median slope}, \text{median land use}).$$

This produces a one-dimensional slice through the original high-dimensional prediction surface.

The main advantages of conditional effect plots are:

- they are simple to interpret;
- they show the direction and shape of the model relationship;
- they reveal nonlinear patterns and thresholds.

However, the interpretation depends strongly on the chosen conditioning values. Different values of X_{-j} may produce different curves because predictors often interact.

Therefore, conditional effect plots describe:

How the model behaves for a specific set of conditions.

They do not represent the average effect of a predictor across the entire dataset.

```
# Create a sequence of Glucose values
set.seed(123)
glucose_seq <- seq(min(train_data$Glucose), max(train_data$Glucose),length.out = 100)

# Create a typical observation
newdata_cond <- data.frame(Glucose = glucose_seq)

# Fix all other predictors at their median values
for (var in names(train_data)[names(train_data) != "Outcome"]) {

  if (var != "Glucose") {
    newdata_cond[[var]] <- median(train_data[[var]], na.rm = TRUE)
  }
}
head(newdata_cond)
```

```
##     Glucose Pregnancies BloodPressure SkinThickness Insulin BMI
## 1  0.000000           3            70            23      37  32
## 2  2.010101           3            70            23      37  32
## 3  4.020202           3            70            23      37  32
## 4  6.030303           3            70            23      37  32
## 5  8.040404           3            70            23      37  32
## 6 10.050505           3            70            23      37  32
##   DiabetesPedigreeFunction Age
## 1                     0.38  29
## 2                     0.38  29
## 3                     0.38  29
## 4                     0.38  29
## 5                     0.38  29
## 6                     0.38  29
```

```
# Predict probability of Outcome = 1
newdata_cond$predicted_prob <- predict(rf_fit,
                      newdata = newdata_cond,type = "prob")[, "1"]

#par(mar=c(5,5,1,1))  #alternatively
#plot(newdata_cond$Glucose, newdata_cond$predicted_prob, type="l", las=1,
#     xlab="Glucose", ylab="Predicted probability of diabetes")

# Plot conditional effect
cond_plot = ggplot(newdata_cond,aes(x = Glucose,y = predicted_prob)) +
  geom_line() + labs(x = "Glucose", y = "Predicted probability of diabetes") +
  theme_bw()
cond_plot
```

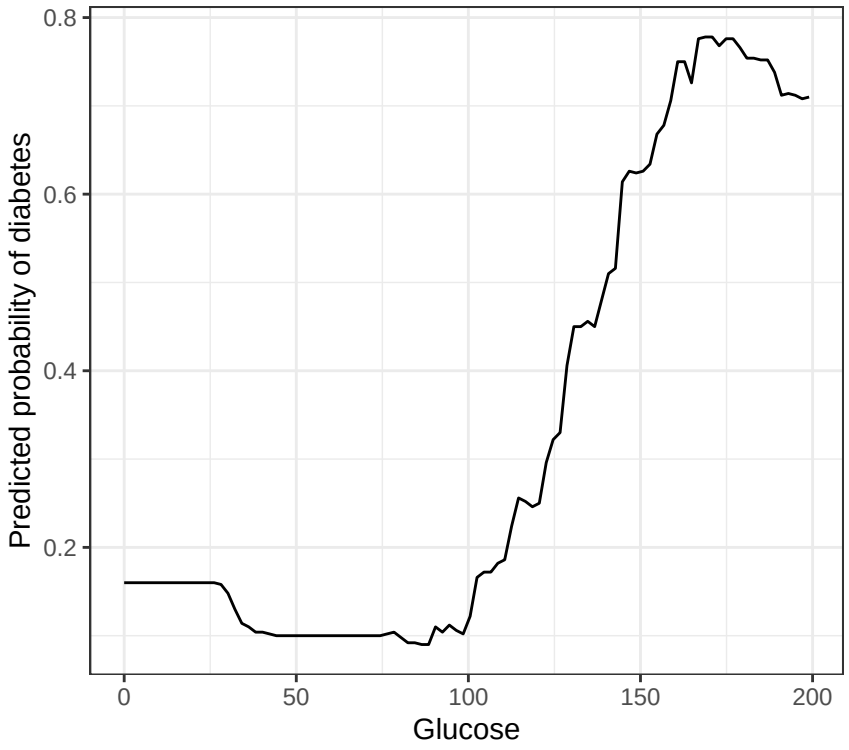


Figure 1: Conditional effect of glucose on diabetes

Unlike classical regression models, tree-based classification methods generally do not provide analytical standard errors for predicted probabilities. Therefore, uncertainty around predictions or effect plots can be estimated using resampling approaches, such as bootstrap refitting of the model. In this case, bootstrap-based confidence intervals are obtained from the empirical distribution of predictions generated by the refitted models.

3.1.1 Bootstrap RandomForest for Confidence Intervals

```
# Bootstrap RandomForest

set.seed(123)
rf_fit_list <- replicate(25, {
  randomForest(Outcome ~., data=train_data[sample(nrow(train_data), replace=T),],
               ntree = 500, importance=TRUE)
  }, simplify=F)


pred_boots <- sapply(rf_fit_list, function(x) predict(x, newdata=newdata_cond,type = "prob")[, "1"])
CIs <- apply(pred_boots, 1, quantile, c(0.025, 0.975))
newdata_cond$lower <- CIs[1,]
newdata_cond$upper <- CIs[2,]

cond_plot2 = ggplot(newdata_cond,aes(x = Glucose,y = predicted_prob)) +
  geom_line(linewidth = 1) +
  geom_line(aes(y = lower), linetype = "dashed") +
  geom_line(aes(y = upper), linetype = "dashed") +
```

```
  labs(x = "Glucose", y = "Predicted probability of diabetes") +
  theme_bw()
cond_plot2
```

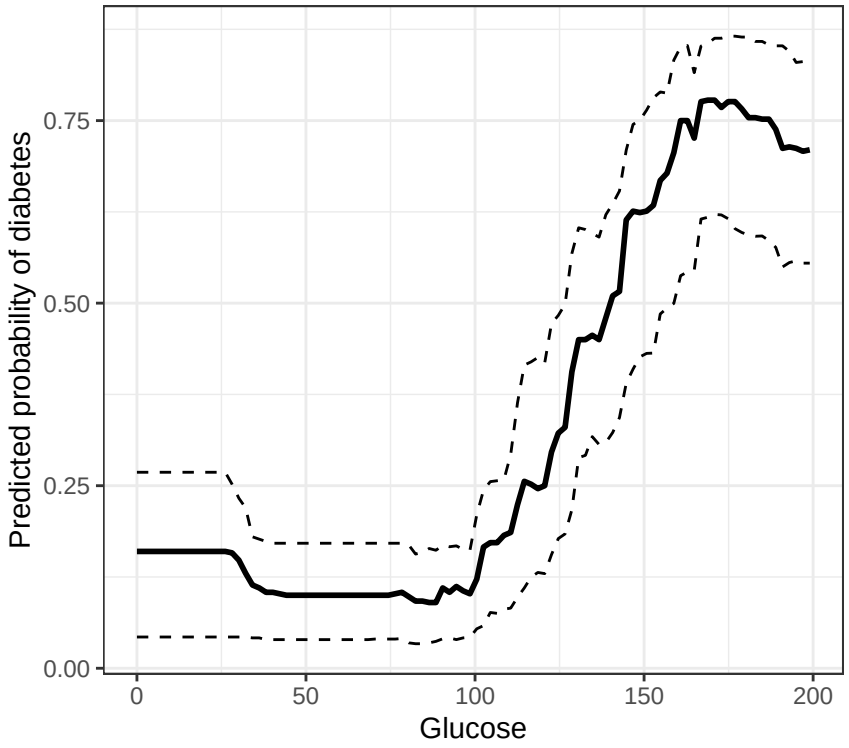


Figure 2: Conditional effect of glucose on diabetes including their CIs

3.2 Individual conditional expectation (ICE) plots

A conditional effect plot displays the relationship between one predictor and the model prediction while keeping the remaining predictors fixed at typical values. However, the resulting curve depends on the particular conditioning values chosen and may not represent the behaviour of all observations.

Individual Conditional Expectation (ICE) plots overcome this limitation by displaying a separate prediction curve for each observation in the data set. Instead of fixing the remaining predictors at a single value, each observation retains its own values for the other predictors while the predictor of interest is varied across its range.

For a predictor x_j, the ICE curve for observation i is:

$$ICE_i(x_j) = f(x_j, \mathbf{x}_{i,-j}),$$

where x_j is varied while all other predictors $\mathbf{x}_{i,-j}$ remain fixed.

Consequently, an ICE plot consists of one prediction curve for every observation in the data set. Differences among the curves indicate that the effect of the predictor depends on the values of other predictors, revealing possible interactions and heterogeneous responses. Large differences between ICE curves suggest interactions between the predictor of interest and other variables in the model.

Compared with a conditional effect plot, ICE plots provide a much richer view of the fitted model because they preserve observation-specific information instead of relying on a single representative conditioning value.

3.2.1 ICE manually

```
par(mar=c(5,5,1,1))

# create empty plot
plot(
  glucose_seq,
  rep(NA, length(glucose_seq)),
  type = "n",
  xlab = "Glucose",
  ylab = "Predicted probability of diabetes",
  ylim = c(0,1)
)

for (i in 1:nrow(train_data)){
each.datum <- train_data[
  rep(i, length(glucose_seq)), !(names(train_data) %in% "Outcome"), drop = F
  ]
each.datum$Glucose <- glucose_seq
lines(each.datum$Glucose, predict(rf_fit, newdata = each.datum, type = "prob")[, "1"],
      col=rgb(0, 0, 1, 0.2)) # rgb(red, green, blue, alpha)
}
```

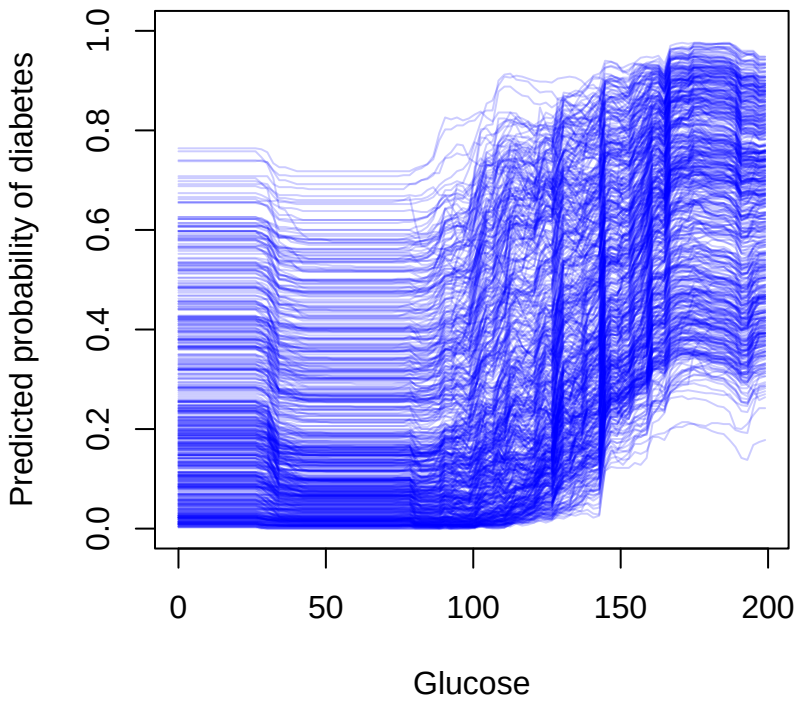


Figure 3: ICE plot (or conditional plot for all cases) of effect of glucose on diabetes, with each line representing an observed combination of the other predictors. This is the manual way to make an ICE plot.

```
datum1 <- train_data[
  rep(1, length(glucose_seq)), !(names(train_data) %in% "Outcome"), drop = F
```

```
  ]
datum1$Glucose <- glucose_seq

datum2 <- train_data[
  rep(2, length(glucose_seq)), !(names(train_data) %in% "Outcome"), drop = F
  ]
datum2$Glucose <- glucose_seq

head(datum1) # first row to see how the data changes
```

```
##       Pregnancies   Glucose BloodPressure SkinThickness Insulin  BMI
## 415             0  0.000000            60            35     167 34.6
## 415.1           0  2.010101            60            35     167 34.6
## 415.2           0  4.020202            60            35     167 34.6
## 415.3           0  6.030303            60            35     167 34.6
## 415.4           0  8.040404            60            35     167 34.6
## 415.5           0 10.050505            60            35     167 34.6
##       DiabetesPedigreeFunction Age
## 415                      0.534  21
## 415.1                    0.534  21
## 415.2                    0.534  21
## 415.3                    0.534  21
## 415.4                    0.534  21
## 415.5                    0.534  21
```

```
head(datum2) # second row to see how the data changes
```

```
##       Pregnancies   Glucose BloodPressure SkinThickness Insulin  BMI
## 463             8  0.000000            70            40      49 35.3
## 463.1           8  2.010101            70            40      49 35.3
## 463.2           8  4.020202            70            40      49 35.3
## 463.3           8  6.030303            70            40      49 35.3
## 463.4           8  8.040404            70            40      49 35.3
## 463.5           8 10.050505            70            40      49 35.3
##       DiabetesPedigreeFunction Age
## 463                      0.705  39
## 463.1                    0.705  39
## 463.2                    0.705  39
## 463.3                    0.705  39
## 463.4                    0.705  39
## 463.5                    0.705  39
```

3.2.2 ICE: using existing packages

```
library(iml) #Interpretable machine learning

#prediction wrapper
predict_prob <- function(model, newdata) {
  predict(model, newdata = newdata, type = "prob")[, "1"]
}

#Predictor object
predictor <- Predictor$new(rf_fit, data = train_data[, !names(train_data) %in% "Outcome"],
  y = train_data$Outcome, predict.function = predict_prob)
```

```r
#ICE plot
ice_plot <- FeatureEffect$new(predictor, feature = "Glucose",method = "ice")
plot(ice_plot)
```

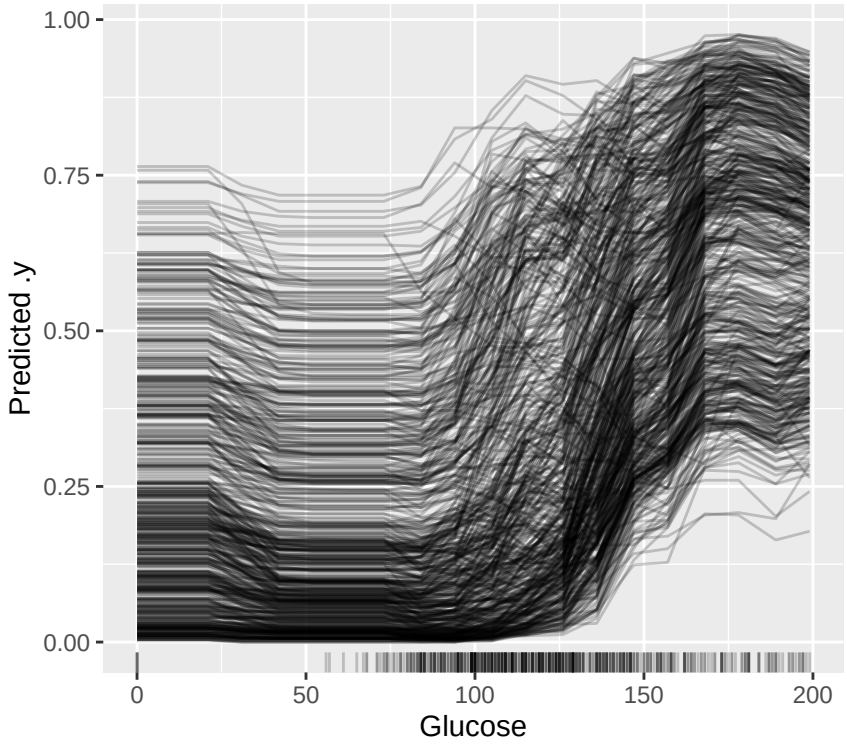


Figure 4: ICE plot (or conditional plot for all cases) using the iml package in R

3.3 Partial dependence plots (PDPs)

While ICE plots display one prediction curve for each observation, PDPs average these curves to obtain a single global effect of the predictor

For a predictor X_j, the partial dependence function is:

$$PD(X_j) = E_{\mathbf{X}_{-j}}\left[f(X_j, \mathbf{X}_{-j})\right],$$

where $\mathbf{X}_{-j}$ represents all predictors excluding X_j.

In practice, the expectation is estimated by averaging predictions over the observed data:

$$PD(x_j) = \frac{1}{n}\sum_{i=1}^{n} f(x_j, \mathbf{x}_{i,-j}),$$

and the calculation is performed by:

1. selecting a value of X_j;
2. replacing X_j with this value for all observations;
3. predicting the response using the fitted model;
4. averaging the resulting predictions.

The resulting curve describes the average relationship between X_j and the model prediction.

For example, for a flood prediction model, a PDP for rainfall shows the average change in predicted flood probability as rainfall increases across all catchments represented in the data.

The main advantages of PDPs are:

- they provide a global summary of predictor effects;
- they are applicable to any prediction model;
- they reveal nonlinear relationships and threshold behaviour.

However, PDPs have an important limitation. They may evaluate combinations of predictors that are rare or unrealistic in the observed data.

For example, increasing rainfall while simultaneously keeping all other predictors unchanged may create environmental conditions that do not occur in reality.

Therefore, PDPs should be interpreted carefully when predictors are strongly correlated.

3.3.1 Manual Pdp plot

Note that in practice, instead of 537 observations (*nrow(train_data)*), we now have 100 × 537 observations to predict to.

```
pdp.mat <- matrix(NA, nrow=nrow(train_data), ncol=length(glucose_seq)) #Matrix for ICE preds
for (i in 1:nrow(train_data)){
  each.obs <- cbind.data.frame("Glucose"=glucose_seq,
                               train_data[i, !(names(train_data) %in% "Outcome"), drop=F])
  pdp.mat[i, ] <- predict(rf_fit, newdata = each.obs, type = "prob")[, "1"]
}
# Partial dependence = average of ICE curves
pdp_manual <- colMeans(pdp.mat)

#plot
par(mar=c(5,5,1,1))

# create empty plot
plot(
  glucose_seq,
  rep(NA, length(glucose_seq)),
  type = "n",
  xlab = "Glucose",
  ylab = "Predicted probability of diabetes",
  ylim = c(0.1, 0.7)
)
lines(glucose_seq, pdp_manual)
```

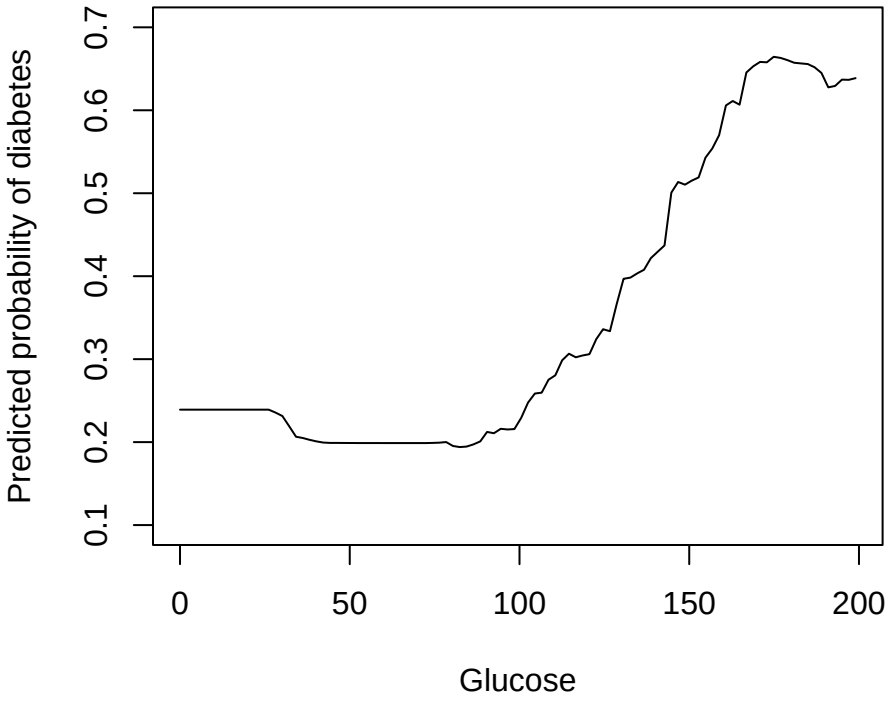


Figure 5: Pdp plot: manually

3.3.2 Pdp plot: using existing R package

```r
library(pdp)

# partial dependence plot
pdp_plot <- pdp::partial(
  object = rf_fit,
  pred.var = "Glucose", # no more than three variables
  train = train_data,
  which.class = "1",    # Probability of the positive class
  prob = TRUE,
  plot = TRUE,
  ice = FALSE,
  center = TRUE,
  rug = TRUE
)

pdp_plot
```

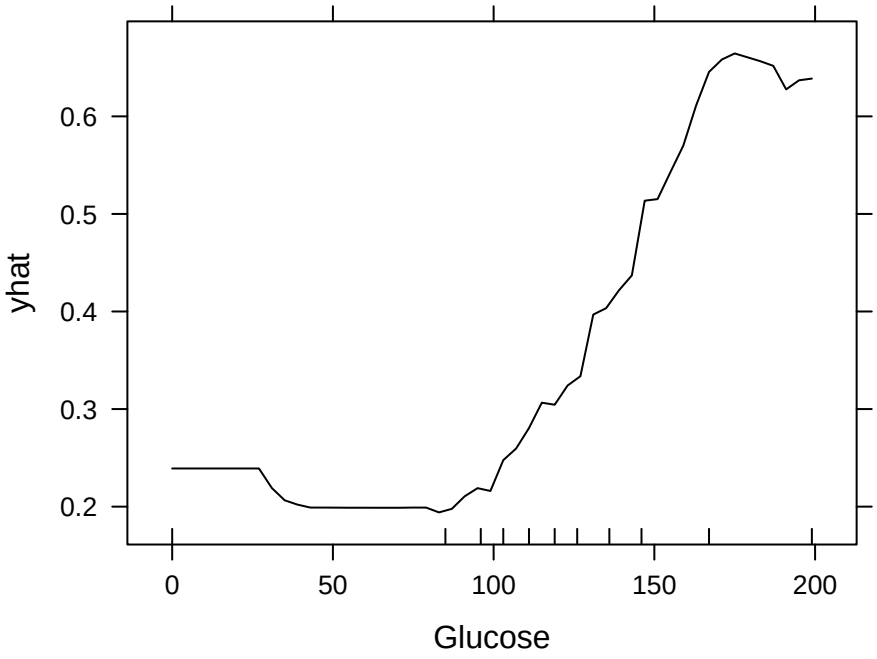


Figure 6: Pdp plot: using pdp package in R

3.4 Conditional effects versus partial dependence

Conditional effect plots and Partial Dependence Plots answer slightly different questions.

A conditional effect plot asks:

How does the model prediction change when one predictor varies under a specific set of conditions?

The remaining predictors are fixed at representative values:

$$\hat{Y} = f(X_j, \mathbf{X}^*_{-j}),$$

where $\mathbf{X}^*_{-j}$ represents selected typical values of the remaining predictors (for example, medians for continuous variables and representative categories for categorical variables).

In contrast, a PDP asks:

What is the average model prediction when one predictor varies across the population?

The remaining predictors are averaged over:

$$PD(X_j) = E_{\mathbf{X}_{-j}}[f(X_j, \mathbf{X}_{-j})].$$

Therefore:

Conditional effect : fix predictors, then predict
Partial dependence : predict for all observations, then average

Because PDPs average predictions over the observed population, they summarize the average model behaviour across the dataset. However, the combinations created by varying one predictor may still be unrealistic when predictors are strongly correlated.

Therefore, PDPs provide a global summary of model behaviour across the dataset, but their interpretation depends on whether the generated predictor combinations are realistic.

Both approaches may become difficult to interpret when strong interactions exist between predictors. In such cases, the effect of one predictor may depend strongly on the values of other predictors.

```
library(gridExtra)
grid.arrange(cond_plot, pdp_plot, nrow = 2)
```

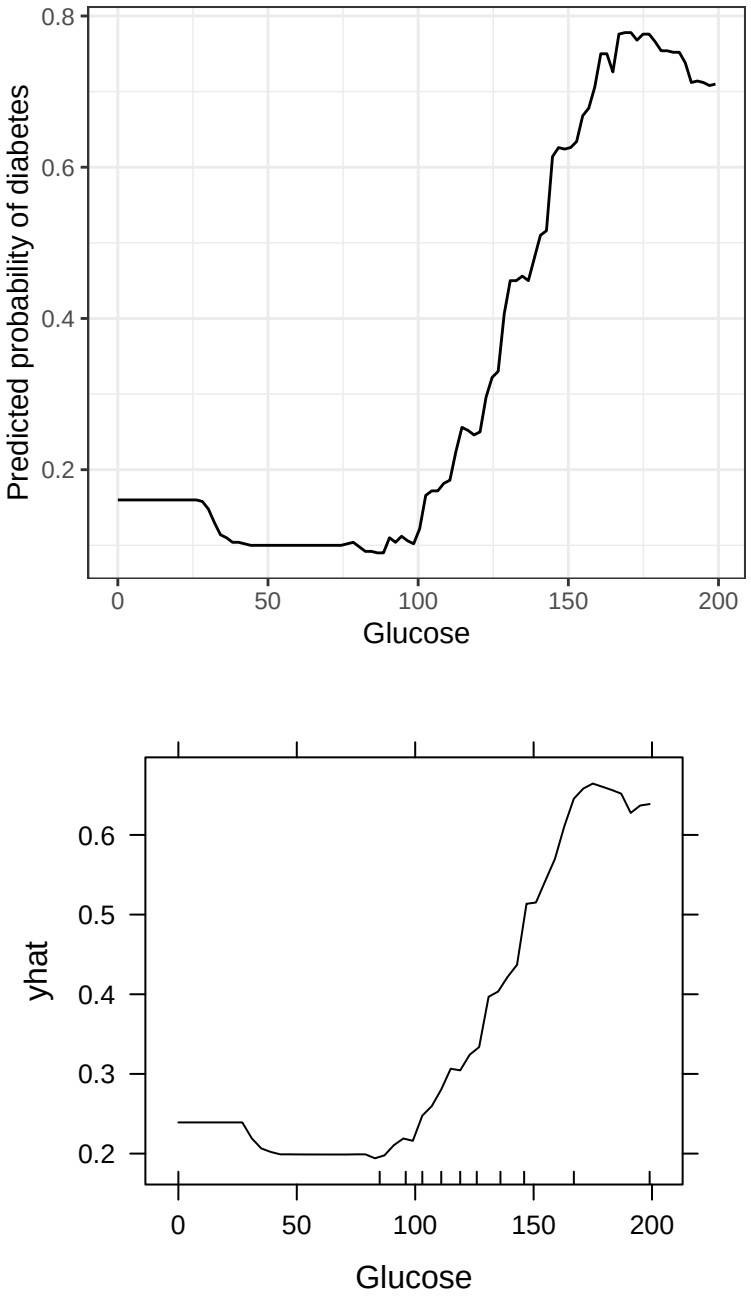


Figure 7: Conditional effect (top) versus Partial dependence (bottom) of glucose

3.5 Accumulated local effects (ALE) plots

Partial dependence plots are useful for understanding the average relationship between a predictor and model predictions. However, PDPs can become misleading when predictors are correlated because they may evaluate combinations of predictor values that are not present in the observed data.

For example, in environmental applications, high elevation and high temperature may rarely occur together. A PDP may still estimate predictions for such unrealistic combinations.

ALE plots overcome this limitation by measuring **local changes in predictions** within regions of the observed predictor space.

Instead of changing a predictor to arbitrary values and averaging predictions, ALE:

1. divides the predictor range into intervals;
2. calculates the change in prediction within each interval;
3. accumulates these local changes to describe the overall predictor effect.

Conceptually, the ALE function can be represented as:

$$ALE(x_j) = \sum \Delta f(x_j),$$

where $\Delta f(x_j)$ represents the local change in prediction caused by a change in predictor x_j.

The main difference between PDP and ALE is summarized below:

Table 1: Comparison of PDP and ALE plots.

Method	Calculation	Main characteristic
PDP	Average predictions over observations	May include unrealistic predictor combinations
ALE	Accumulates local prediction changes	Uses only the observed predictor distribution

Because ALE uses local changes within the observed data distribution, it can be advantageous for environmental datasets where predictors are correlated.

For example, climate variables, topography, and soil properties often exhibit strong dependencies, making ALE plots a useful alternative to PDPs.

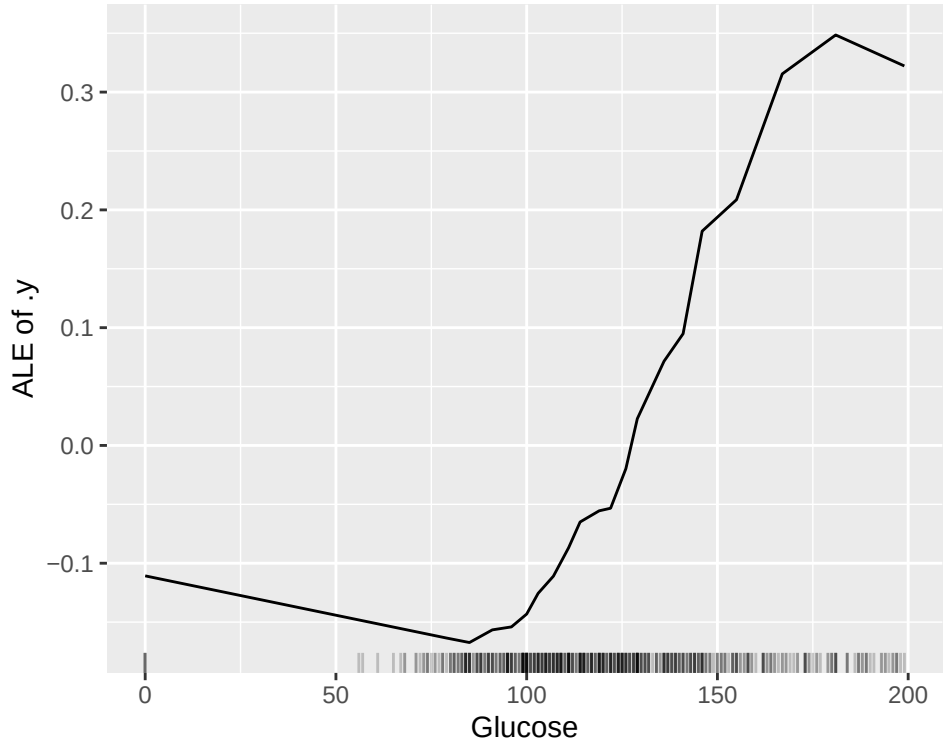
ALE example in R

The `iml` package provides tools for calculating ALE plots for machine learning models.

For classification models, the prediction function must return the predicted probability of the positive class:

3.5.1 Using the "iml" package in R

```
ale_glucose <- FeatureEffect$new(predictor, feature = "Glucose",method = "ale")
plot(ale_glucose)
```




```
# only method="ale", "pdp", and "ice" is allowed
# only one for one var per plot
```

3.5.2 Manual approach

```
# Define bins
n_bins <- 20

glucose_bins <- quantile(train_data$Glucose,probs = seq(0, 1, length.out = n_bins + 1)
)

# Remove duplicated cut points
glucose_bins <- unique(glucose_bins)

# Store local effects
ale_effect <- numeric(length(glucose_bins)-1)

for (k in 1:(length(glucose_bins)-1)) {

  # observations in this interval
  ind <- which(
    train_data$Glucose >= glucose_bins[k] &
    train_data$Glucose <= glucose_bins[k+1]
  )

  if(length(ind) > 0){

    lower_data <- train_data[ind,
      !(names(train_data) %in% "Outcome"),
```

```
      drop = FALSE
    ]

    upper_data <- lower_data

    # move Glucose to lower and upper boundary
    lower_data$Glucose <- glucose_bins[k]
    upper_data$Glucose <- glucose_bins[k+1]

    pred_lower <- predict(rf_fit,newdata = lower_data,type = "prob")[, "1"]
    pred_upper <- predict(rf_fit,newdata = upper_data,type = "prob")[, "1"]

    # average local change
    ale_effect[k] <- mean(pred_upper - pred_lower)
  }
}

# Accumulate local effects
ale_values <- cumsum(ale_effect)

# Center ALE around zero
ale_values <- ale_values - mean(ale_values)

# Plot ALE
plot(glucose_bins[-1],ale_values, type="l", lwd=3, xlab="Glucose",
  ylab="Accumulated local effect")
abline(h=0, lty=2)
```

The resulting curve describes how the predicted probability of the positive class changes as Glucose varies,

while considering only changes supported by the observed data.

4. Uncertainty in model interpretation

Effect plots provide a useful summary of complex models, but the estimated relationship is not exact. There is uncertainty associated with the displayed effect.

Two main sources of uncertainty should be considered:

- **Conditioning uncertainty**: the effect may change depending on the values of the other predictors that are fixed or averaged over;
- **Model uncertainty**: the fitted model itself is uncertain because it is estimated from a finite sample of observations.

For conditional effect plots, the first source of uncertainty is particularly important.

A conditional effect plot represents:

$$\hat{Y} = f(X_j, \mathbf{X}^*_{-j}),$$

where $\mathbf{X}^*_{-j}$ represents the chosen conditioning values.

Changing $\mathbf{X}^*_{-j}$ may produce a different relationship because predictors can interact.

For example, the effect of rainfall on flood risk may differ between small and large catchments:

$$f(\text{rainfall}|\text{small catchment}) \neq f(\text{rainfall}|\text{large catchment}).$$

Therefore, a single conditional curve does not represent all possible environmental conditions.

The second source of uncertainty arises because the model is estimated from observed data.

Different training samples may produce slightly different models and therefore different predicted relationships.

In practice, uncertainty is often represented by confidence bands around an effect curve:

$$\hat{f}(X_j) \pm [\hat{f}_{lower}(X_j), \hat{f}_{upper}(X_j)].$$

However, many interpretation methods display only one source of uncertainty, usually the variation caused by conditioning values or the variability of the estimated predictions.

Therefore, effect plots should be interpreted as summaries of model behaviour, rather than exact descriptions of the underlying environmental process.

5. Variable importance

Effect plots describe the relationship between a predictor and model predictions. However, they do not directly indicate which predictors are the most influential in the model.

Variable importance measures address this question by ranking predictors according to how strongly they influence model predictions or model performance.

Two broad categories of importance measures exist:

- **Model-specific importance**: measures that use information from the internal structure of a particular model;
- **Model-agnostic importance**: measures that can be applied to any prediction model.

5.1 Model-specific importance

Model-specific importance measures are based on the characteristics of the model itself.

Different statistical models therefore have different importance measures.

For example:

- linear models: importance may be assessed using regression coefficients or explained variance;
- analysis of variance: importance may be related to sums of squares;
- CART-based models: importance is commonly based on the reduction in impurity produced by splits involving each predictor.

For a classification tree, a split is selected because it reduces node impurity:

$$\Delta G = G(\text{parent}) - G(\text{split}).$$

Predictors that repeatedly produce large impurity reductions across many trees are assigned higher importance.

For Random Forests and boosting models, these split-based measures can be summarized across all trees in the ensemble.

The advantage of model-specific importance is that it is computationally efficient and directly linked to the model construction.

However, it cannot be compared directly between different model classes because the definition of importance depends on the fitted algorithm.

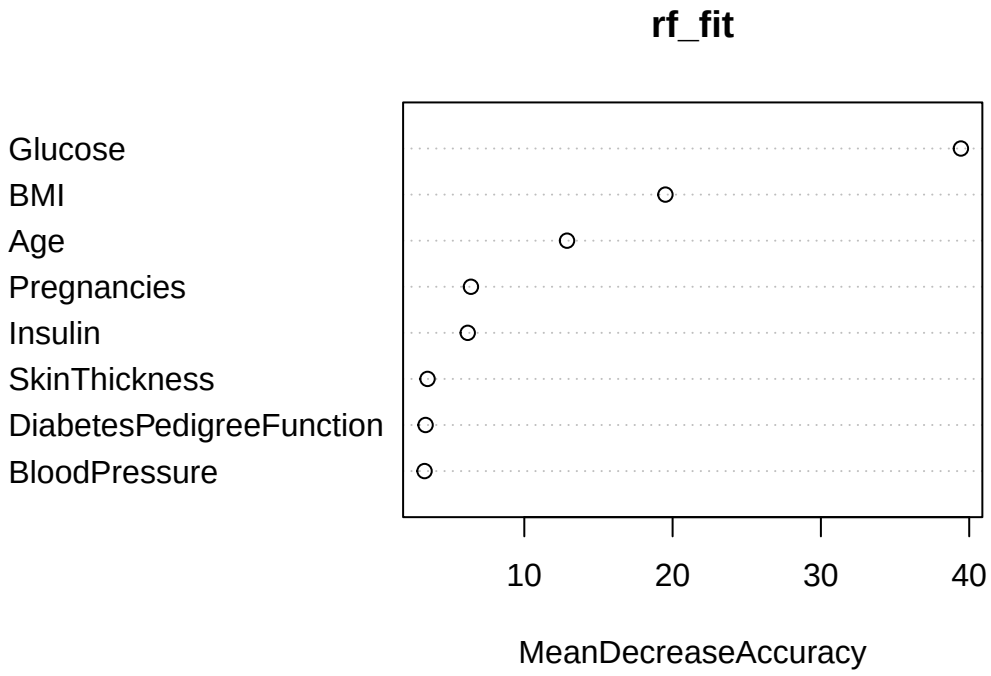
5.1.1 RandomForest example

The randomForest package calculates variable importance during model fitting by setting "importance = TRUE".

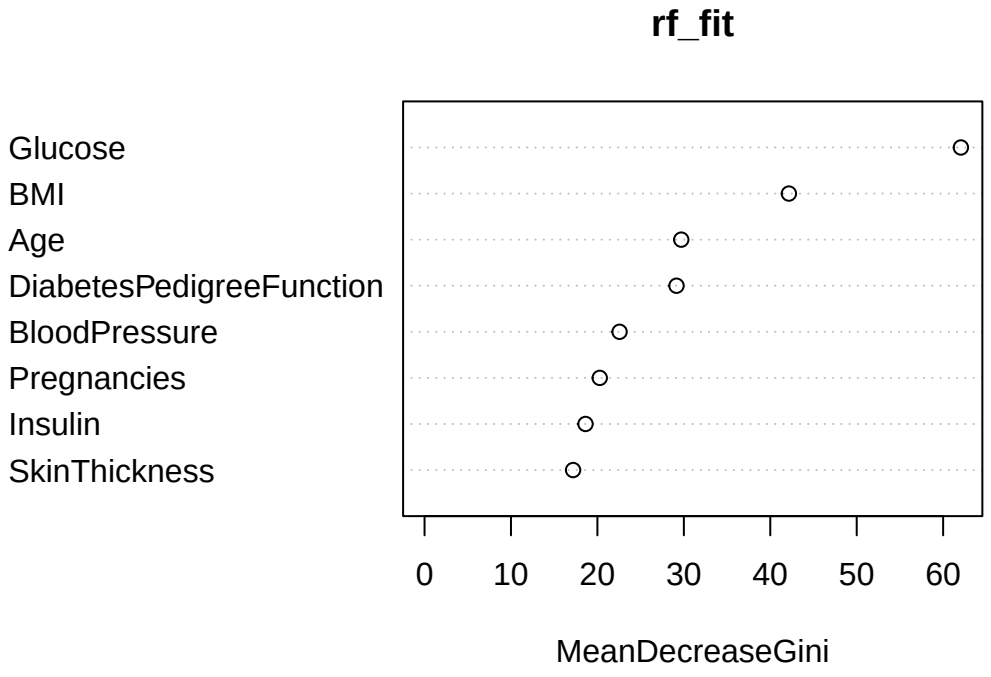
```
# Extract variable importance
randomForest::importance(rf_fit)
```

```
##                                0          1 MeanDecreaseAccuracy
## Pregnancies                6.960699  0.7502564             6.401125
## Glucose                   31.023671 30.4403281            39.440392
## BloodPressure              3.381981  0.6236178             3.273740
## SkinThickness              4.271716 -0.2013018             3.475019
## Insulin                    7.217119  0.4910227             6.184884
## BMI                       10.937649 18.2257999            19.514937
## DiabetesPedigreeFunction  2.387901  2.5650280             3.341208
## Age                       10.127304  5.9777400            12.880283
##                           MeanDecreaseGini
## Pregnancies                       20.26939
## Glucose                           62.05559
## BloodPressure                     22.56097
## SkinThickness                     17.18462
## Insulin                           18.62340
## BMI                               42.15505
## DiabetesPedigreeFunction          29.15019
## Age                               29.70118
```

```
# Plot variable importance
randomForest::varImpPlot(rf_fit, type=1)
```




```
randomForest::varImpPlot(rf_fit, type=2)
```



5.1.2 XGBoost example

XGBoost provides importance measures based on how often and how effectively variables are used in tree construction.

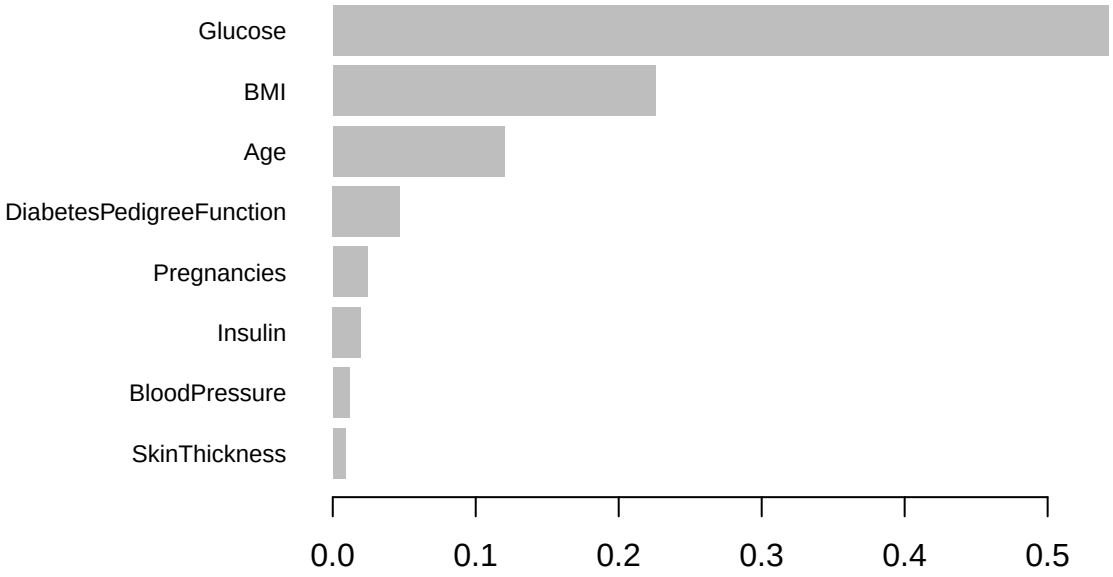
The main measures include:

- **Gain**: average improvement in the objective function from splits using the predictor;
- **Cover**: number of observations affected by those splits;
- **Frequency**: how often the predictor is used.

```
library(xgboost)
set.seed(123)

X_train <- as.matrix(train_data[, !names(train_data) %in% "Outcome"])
X_test <- as.matrix(test_data[, !names(test_data) %in% "Outcome"])

y <- train_data$Outcome

xgb_model <- xgboost(data = X_train,label = y,nrounds = 500,max_depth = 2,eta = 0.01,
  objective="binary:logistic")

imp_table <- xgb.importance(model = xgb_model)
imp_table
```

```
##                     Feature        Gain       Cover  Frequency
##                      <char>       <num>       <num>      <num>
## 1:                  Glucose 0.542467874 0.379966533 0.34000000
## 2:                      BMI 0.225584680 0.254097804 0.24266667
## 3:                      Age 0.120245367 0.129139294 0.12733333
## 4: DiabetesPedigreeFunction 0.047159410 0.094561073 0.11333333
## 5:              Pregnancies 0.024378355 0.061518894 0.04933333
## 6:                  Insulin 0.019811003 0.036136357 0.05666667
## 7:            BloodPressure 0.011595279 0.039200791 0.03600000
## 8:            SkinThickness 0.008758031 0.005379253 0.03466667
```

```
xgb.plot.importance(imp_table)
```



5.2 Model-agnostic importance

Model-agnostic importance measures do not depend on how the model was fitted. They can therefore be applied to any prediction algorithm, including CART, Random Forests, boosting, and neural networks.

The general idea is to quantify how strongly a predictor is related to the model's predictions or predictive performance.

Different model-agnostic methods measure this relationship in different ways:

- **Permutation importance** measures how much predictive performance changes when information about a predictor is disrupted;
- **Feature Importance Ranking Measure (FIRM)** measures the variability of model predictions associated with changes in a predictor;
- **Shapley-value-based approaches** quantify how much a predictor contributes to individual predictions.

These methods are flexible because they are not tied to a specific modelling algorithm.

5.2.1 Permutation

Permutation importance measures the influence of a predictor by randomly shuffling its values and evaluating how much the model performance changes. A predictor is considered important when permuting its values causes a large decrease in predictive performance, because the model loses access to information that was useful for prediction.

```
#install.packages("pak")
#install.packages("yardstick")
```

```r
pak::pak("koalaverse/vip")
pak::pak("bgreenwell/fastshap@main")

library(vip)

# Get importance scores
vi_scores1 <- vi(rf_fit, method="model")     # Model-specific, defualt
#vi_scores2 <- vi(rf_fit, method="permute")  # Permutation
#vi_scores3 <- vi(rf_fit, method="shap")     # Shapley values
#vi_scores4 <- vi(rf_fit, method="firm")     # Variance-based

vi_scores2 <- vi(
  rf_fit,
  method = "permute",
  train = train_data[, !names(train_data) %in% "Outcome"],
  target = train_data$Outcome,
  metric = "accuracy",
  pred_wrapper = function(object, newdata) {
  predict(object, newdata, type = "class")
  }
)
#vip(vi_scores2)


permute <- vi_permute(
  object = rf_fit,
  feature_names = colnames(train_data)[!colnames(train_data) %in% "Outcome"],
  train = train_data[, !names(train_data) %in% "Outcome"],
  target = train_data$Outcome,
  metric = "accuracy",
  pred_wrapper = function(object, newdata) {
    predict(object, newdata = newdata, type = "class")
  }
)
permute[order(permute$Importance, decreasing = TRUE),]
```

```
##                   Variable Importance
## 2                  Glucose 0.20111732
## 6                      BMI 0.11545624
## 8                      Age 0.06331471
## 7 DiabetesPedigreeFunction 0.05772812
## 1              Pregnancies 0.02979516
## 5                  Insulin 0.02793296
## 4            SkinThickness 0.02420857
## 3            BloodPressure 0.02234637
```

```r
vip(permute)
```

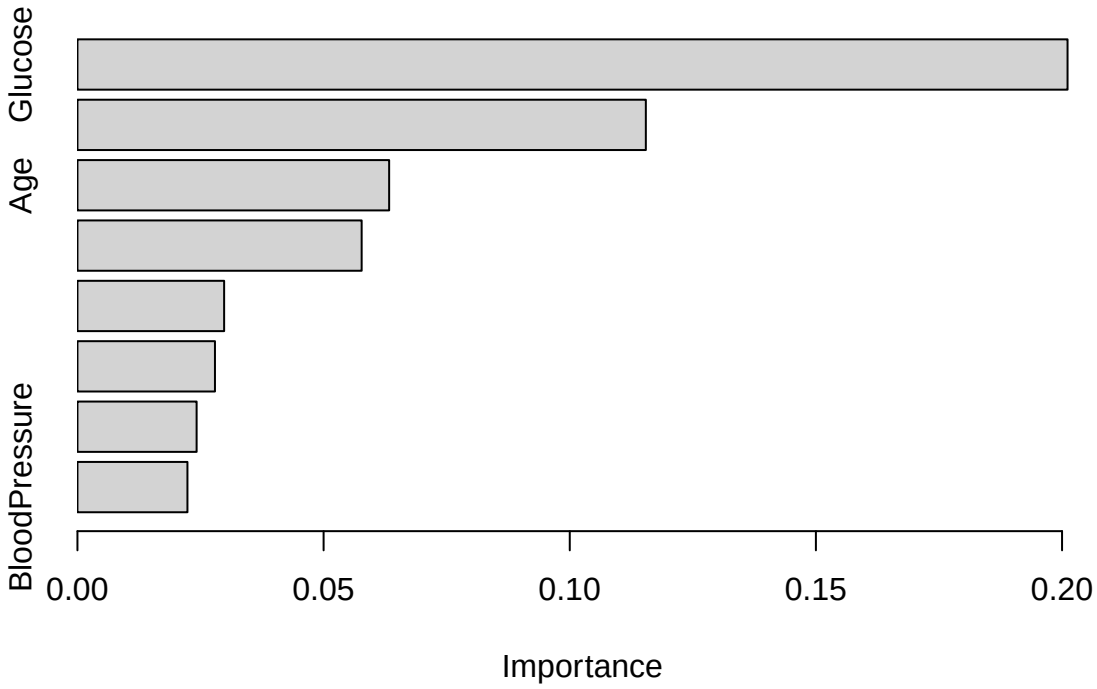


Figure 8: Permutation importance

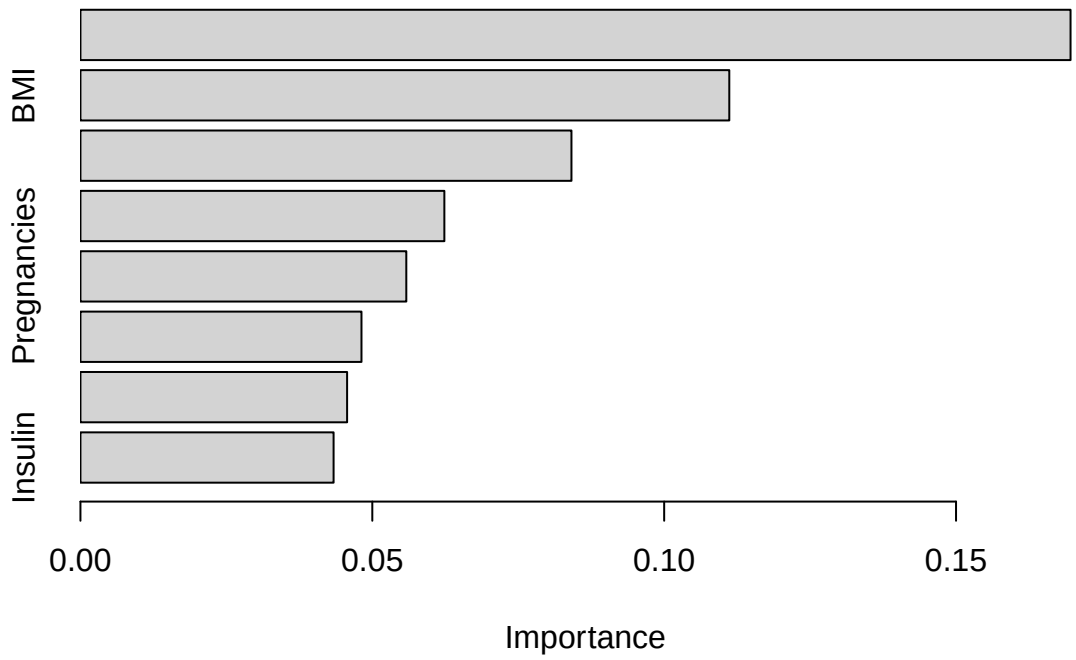
5.2.2 SHAP

SHAP values provide local explanations by quantifying how much each predictor contributes to an individual prediction. A global SHAP importance measure is usually obtained by averaging the absolute SHAP values for each predictor across all observations.

```
vi_scores3 <- vi(
  rf_fit,
  method = "shap",
  train = train_data[, !names(train_data) %in% "Outcome"],
  target = train_data$Outcome,
  metric = "accuracy",
  pred_wrapper = function(object, newdata) {
    predict(object, newdata, type = "class")
  }
)
#vip(vi_scores3)

shap <- vi_shap(rf_fit, train = train_data[, !names(train_data) %in% "Outcome"],
  pred_wrapper = function(object, newdata)
  predict(object, newdata, type = "prob")[, "1"]
)

shap[order(shap$Importance, decreasing = TRUE),]
```

```
##                   Variable Importance
## 2                  Glucose 0.16961639
## 6                      BMI 0.11115456
## 8                      Age 0.08411546
## 7 DiabetesPedigreeFunction 0.06234264
## 1              Pregnancies 0.05582495
## 3            BloodPressure 0.04814525
## 4            SkinThickness 0.04568343
## 5                  Insulin 0.04336685
```

```
vip(shap)
```



5.2.3 FIRM

Feature Importance Ranking Measure (FIRM) quantifies predictor importance by examining how much the model predictions vary as the value of a predictor changes while the remaining predictors are held constant.

Predictors that produce greater variability in model predictions receive higher importance scores.

Conceptually:

$$Importance(X_j) \propto SD(\hat{Y}_{X_j}),$$

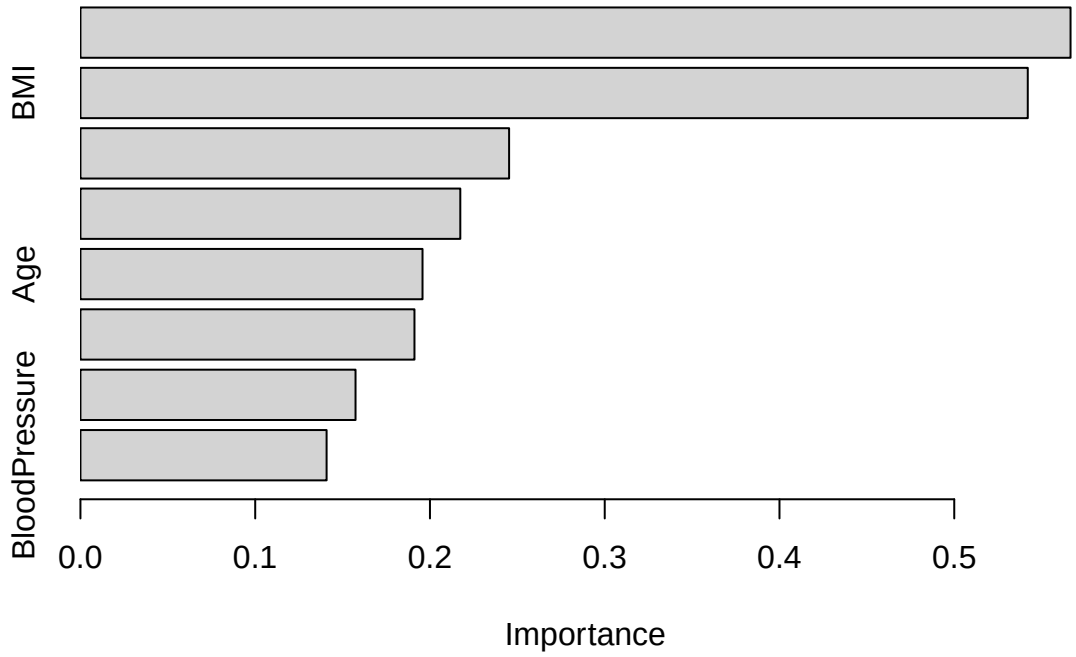
where $\hat{Y}_{X_j}$ represents the model predictions obtained while varying predictor X_j.

```
vi_scores4 <- vi(
  rf_fit,
  method = "firm",
  train = train_data[, !names(train_data) %in% "Outcome"],
```

```
  target = train_data$Outcome,
  metric = "accuracy",
  pred_wrapper = function(object, newdata) {
    predict(object, newdata, type = "class")
  }
)
#vip(vi_scores4)

firm <- vi_firm(rf_fit, feature_names = colnames(train_data)[!colnames(train_data) %in% "Outcome"],
  train = train_data
) # pred_wrapper issue with FIRM

firm[order(firm$Importance, decreasing = TRUE),]
```

```
##                   Variable Importance
## 2                  Glucose  0.5665575
## 6                      BMI  0.5420708
## 1              Pregnancies  0.2452852
## 4            SkinThickness  0.2173702
## 8                      Age  0.1957374
## 7 DiabetesPedigreeFunction  0.1911357
## 5                  Insulin  0.1574070
## 3            BloodPressure  0.1408629
```

```
vip(firm)
```



```r
# comparing with correlation
imp <- data.frame(
  Permutation = permute[, 2],
  FIRM = firm[, 2],
  SHAP = shap[, 2]
)

print(cor(imp))
```

```
##             Permutation      FIRM      SHAP
## Permutation   1.0000000 0.9002534 0.9912753
## FIRM          0.9002534 1.0000000 0.8964987
## SHAP          0.9912753 0.8964987 1.0000000
```

The interpretation is:

- Glucose is the most important predictor because permuting Glucose produces the largest increase in loss.
- Age and BMI have similar importance followed by Insulin. For permutation importance, this indicates that permuting these predictors produces a similar change in model performance.
- All three methods identify similar patterns. Correlations above 0.87 indicate that the variables deemed important by one method tend also to be important according to the others.
- Permutation and SHAP are almost perfectly correlated (0.995). For this Random Forest and dataset, both methods produce nearly identical rankings of predictor importance. This does not mean the methods are mathematically equivalent—it simply means they agree very closely on this particular model.
- FIRM differs slightly. Its correlations with the other methods (0.87–0.89) are still high, but because FIRM measures variability in model predictions rather than the change in prediction performance (permutation) or feature contribution (SHAP), some differences are expected.

5.3 Limitations of variable importance measures

Variable importance measures should be interpreted carefully.

A high importance value indicates that a predictor has a strong influence on the model predictions according to the chosen importance method, but it does not mean that the predictor has a causal effect on the response.

For example, elevation may appear highly important in an ecological model because it is correlated with temperature, precipitation, or vegetation patterns.

In addition, importance can be affected by:

- correlations between predictors;
- the range of observed predictor values;
- the choice of importance method;
- interactions between predictors.

Therefore, variable importance should usually be interpreted together with effect plots.

A useful workflow is:

Variable importance → Identify influential predictors → Effect plots → Understand predictor behaviour

Finally, importance measures based on repeatedly refitting modified models (for example, removing predictors and retraining the model) should be interpreted cautiously.